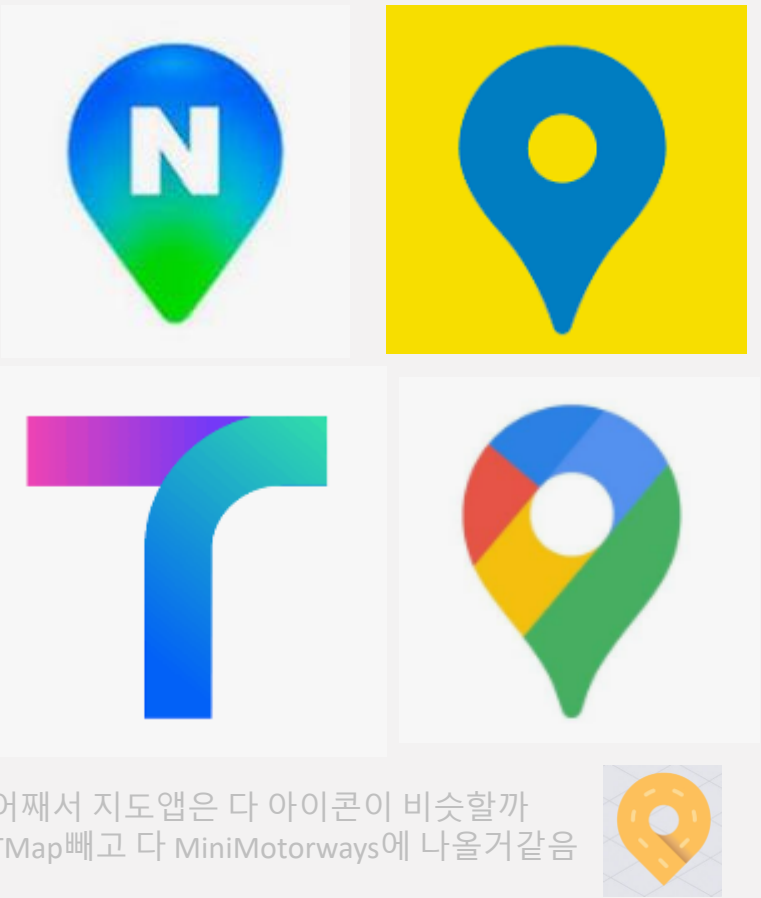
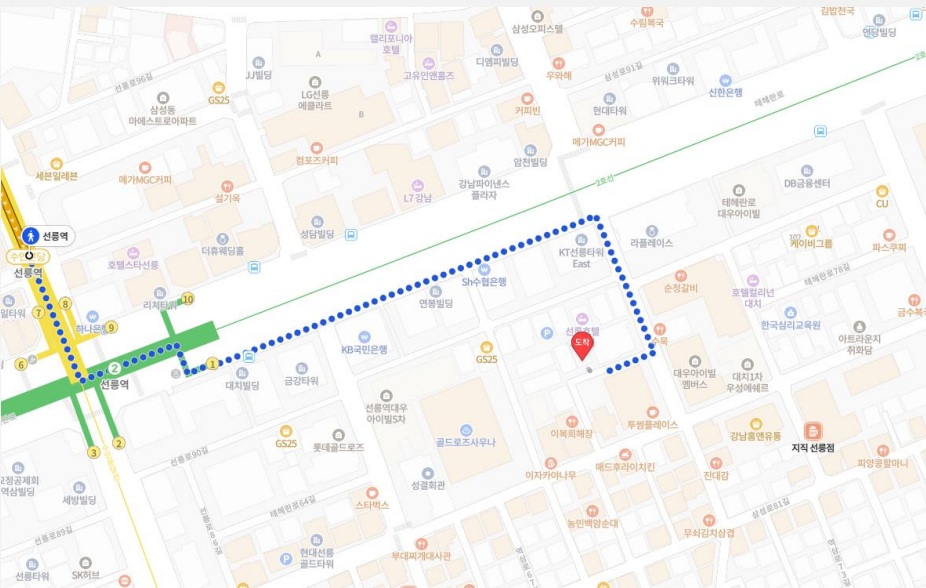
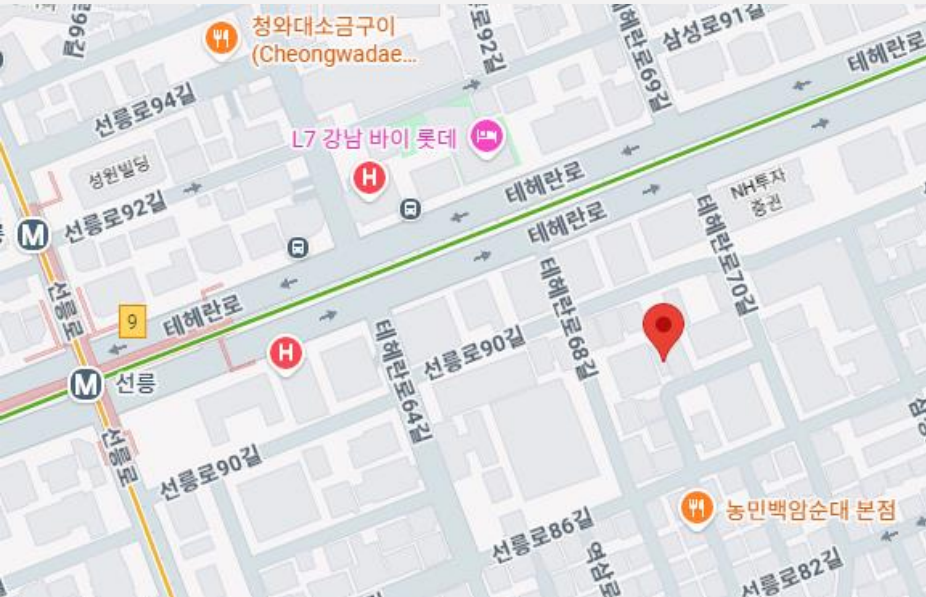


구글맵 설계

지도 렌더링
사용자의 위치를 탐색 및 표기
최적 경로 안내하기



서비스의 규모

01

DAU (일간 활성 사용자)

10억명
얼추 구글급 규모를 상정한 숫자

02

도로 데이터의 규모

수 TB수준의 가공되지 않은 데이터

문제 정의

01

위치 서비스

사용자 단말의 위치를 주기적으로 수신하고 기록한다.

02

경로 안내 서비스

출발지·도착지 사이의 최적 경로와 도착 예정 시각(ETA)을 계산해 제공한다.

03

지도 표시

요청한 영역·줌 레벨의 지도를 화면에 표시한다.

비기능 요구사항

01

정확도

사용자에게 잘못된 경로를 안내해서는 안됨.

02

부드러운 경로 표시

제공되는 경로 안내 용도의 지도는 부드럽게 표시되고 갱신되어야 함.

* 지도가 끊겨보인다거나, 경로가 갱신될때 안정적으로 수신해야함

03

데이터 및 배터리 사용량

클라이언트는 가능한 최소한의 데이터와 배터리를 사용해야함

* 주로 모바일/휴대용 기기이기 때문

*일반적으로 통용되는 가용성 / 규모 확장성 요구사항은 책에서 꾸준히 다루는 주제

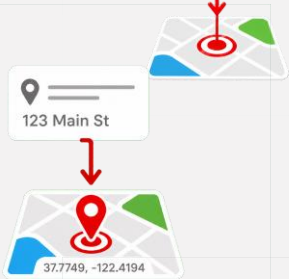
기술 스택 훑아보기



측위 시스템 : 단말의 현재 위치를 어떻게 알아내는가



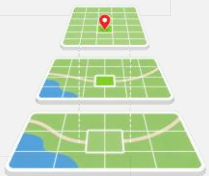
3차원 위치의 2차원 변환 : 지구(구면)를 평면 지도로 펴는 도법



지오코딩 : 사람이 읽는 주소를 좌표로 변환



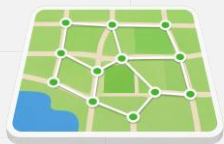
지오해싱 : 좌표를 문자열로 인코딩



지도 표시 : 타일 기반 렌더링



경로 안내를 위한 도로 데이터 : 그래프



계층적 경로 안내 타일 : 그래프를 지역·계층 단위로 분할



측위 시스템



사용자의 좌표를 추정 / 결정하는 시스템

주로 GPS로 위·경도 좌표를 얻습니다.

실내·고층 도심에서는 오차가 커서 보정이 필요합니다.

단말이 주기적으로 위치를 서버에 전송합니다.

책에서는 측위 시스템 까지 설계 범위에 포함한건 아닌 것 같았고
사용자의 디바이스에서 위도 / 경도로 가공해준다고 가정합니다.

3차원 위치의 2차원 변환



구(지구)를 평면 지도로 펴는 과정을 지도 투영법 (도법) 이라고 합니다.
위·경도(구면 좌표)를 평면 x/y 픽셀로 매핑해주게 됩니다.

웹 지도는 웹 메르카토르(EPSG:3857)를 표준으로 사용합니다.

왜 표준이 되었는지는 찾아보니 지구를 정사각형 평면으로 만들수 있고, 비교적 좌표 변환공식이 단순한데 다른것보다 초기 대형지도 서비스들 (구글이라던가)의 사용으로 생태계가 만들어진게 가장 가장 커보입니다
극지방에서 처리가 어렵다는 점이 문제인 것 같은데, 그런 이유에서 주로 85도 위도에서 잘라낸다고 하네요



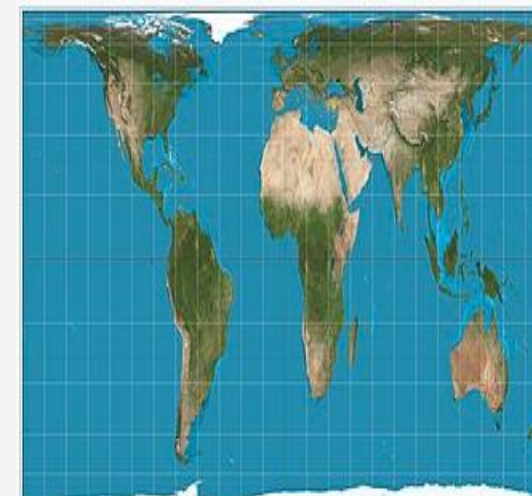
웹 메르카토르



메르카토르



퍼스 퀴쿤설

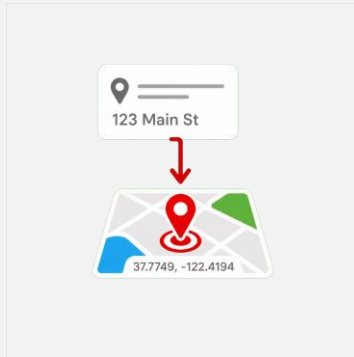


갈 패터스



원켈 트리펠

지오코딩



사람이 읽는 주소와 위·경도 좌표를 서로 변환합니다.

지오코딩: 주소를 좌표 변환

역지오코딩: 좌표를 주소로 변환

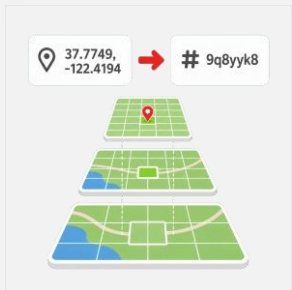
이를 수행하는 방법중에 예시로 든 인터폴레이션(Interpolation)은 다음과 같이 수행됩니다.

서적 내용중 "GIS (Geographic Information System)와 같은 다양한 시스템이 제공하는 데이터를 결합한다." 는 것은

- 도로명 주소 같은게 있으면, 선형으로 이루어진 도로라고 가정하고 선형보간한다거나
- 우편번호의 중심 좌표등...

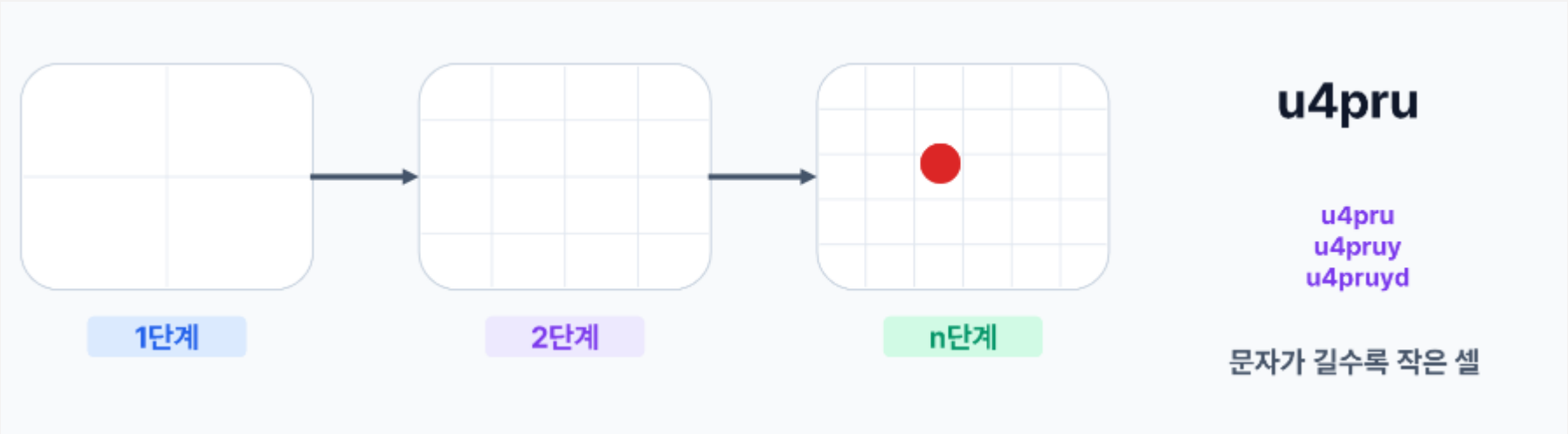
라이브러리에서도 정밀하지 않은 좌표를 반환해줄 수 있어, Google / Azure의 지오코딩 API는 해당 좌표가 어떻게 산출되었는지, 얼마나 정밀할 수 있는지도 함께 반환해준다고 합니다

지오해싱



1장에서 다루었던 지오해싱입니다.

2차원 위·경도를 1차원 문자열로 인코딩합니다.
격자를 재귀적으로 분할하기 때문에
접두사가 같으면 지리적으로 인접 근접 검색에 유리합니다.
특정 영역·반경 내 객체 조회를 인덱스로 빠르게 처리할 수 있게 됩니다.



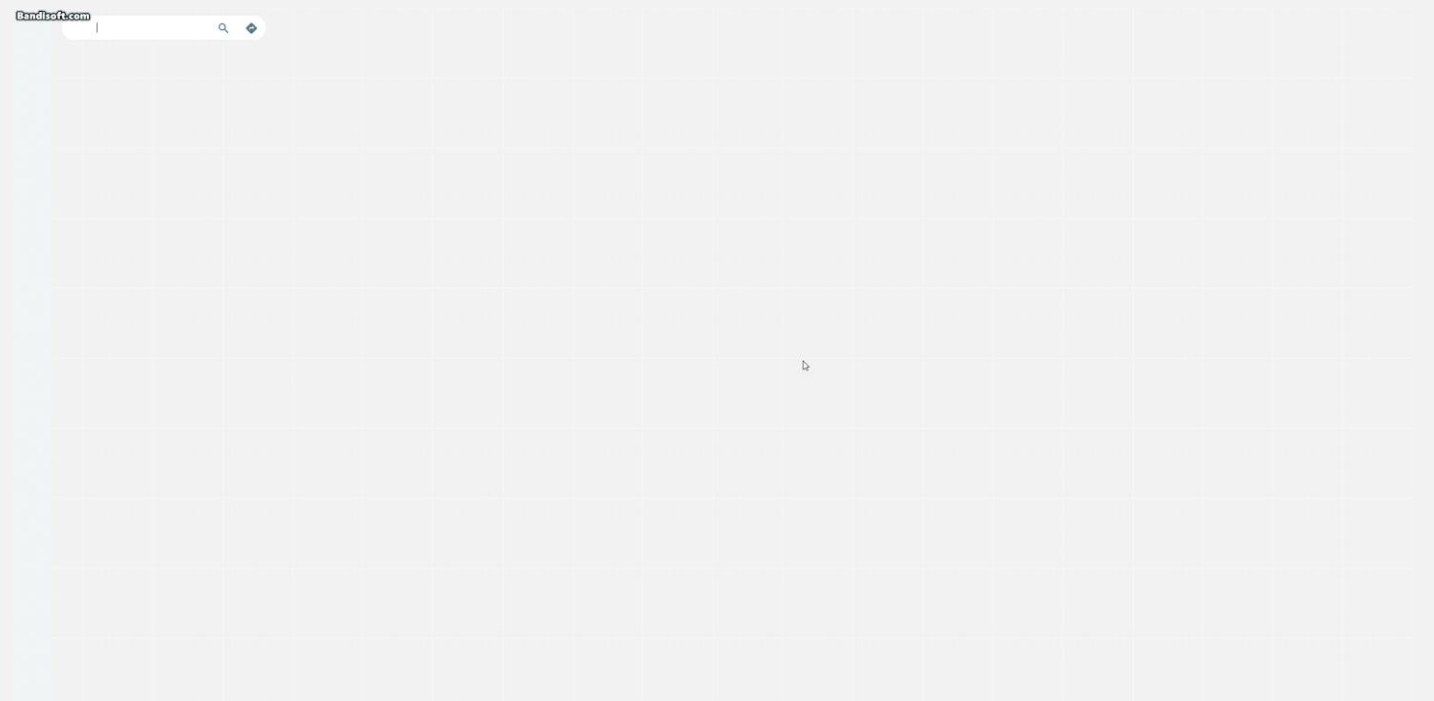
지도 표시 방법



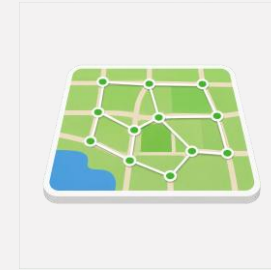
지도를 타일 이미지로 분할해 제공합니다.

지도 확대 수준에 따라 미리 만들어둔 지도 이미지를 제공하게 됩니다.

줌 레벨에 따라 지도의 디테일이 다른것을 보면 알게 될 수도 있는 부분
인터넷 환경이 느리거나 하면 gif의 예시와 같이 타일이 로드되는 흔적도
확인할 수 있습니다.



경로 안내를 위한 도로 데이터 처리

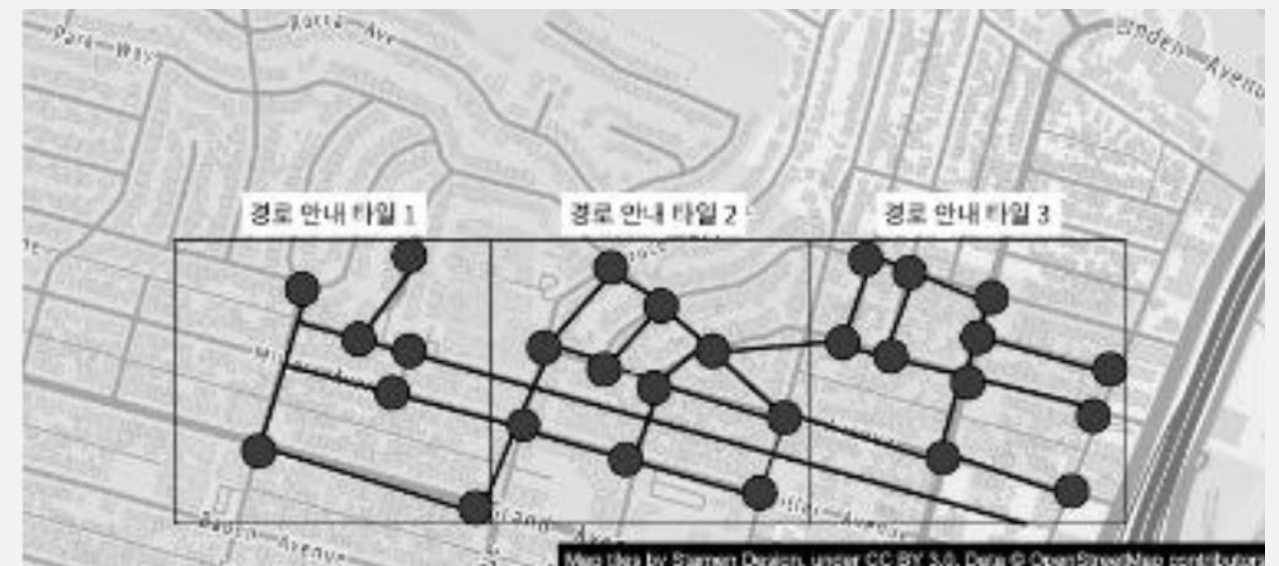


대부분의 경로 탐색 알고리즘은 데이크스트라 (Dijkstra) 알고리즘이나, A* 기반의 경로 탐색 알고리즘의 변종. 세부 알고리즘은 여기서는 다루지 않습니다.

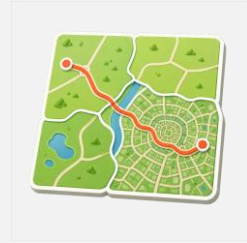
도로 데이터는 결국 node와 edge를 가진 그래프 자료구조로 가공해야 경로 탐색 알고리즘을 수행할 수 있게 됩니다.

전세계 도로망을 하나의 그래프로 표현하면 메모리도 많이 차지하고, 탐색에 불필요한 데이터도 모두 올라가게 됩니다.

이를 해결하기 위해 지오해싱과 유사한 격자 분할 기술을 적용하고 인접 격자의 연결된 참조를 유지하게 구성합니다.



계층적 경로 안내 타일



도로 그래프도 지역 단위 "라우팅 타일"로 분할합니다.

장거리 경로 탐색을 수행할때 정밀한 node까지 탐색하는건 최적해를 찾을 수 있겠지만 많은 시간이 소요되게 됩니다.

대부분 구체성(지역성)이 높은 작은 타일 (골목길)
비교적 넓은 관할구 정도를 커버하는 조금 더 큰 타일 (간선도로)
도시와 주 등을 연결하는 큰 타일 (고속도로) 와 같이 그래프를 구성합니다.

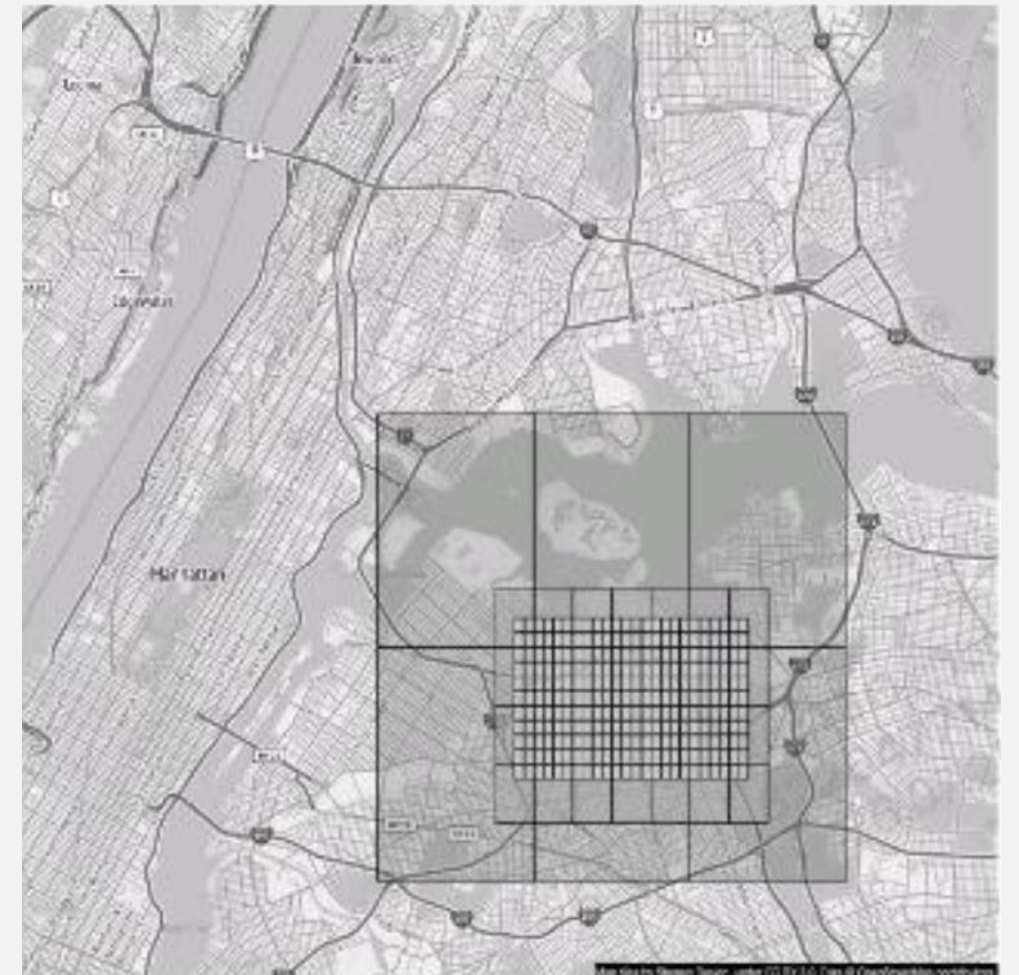


그림 3.6 크기가 서로 다른 경로 안내 타일

개략적 규모 추정

저장소 사용량

지도 사진 정보가 대부분

세계 지도 데이터를 21번 확대해서 볼 수 있으려면 가장 작은 크기의 타일은 4.4조개 필요한
한장의 타일이 256x256픽셀이면 100KB정도, 전부 저장하면 440PB만큼의 저장공간이 필요함.

지구 표면의 90%는 사람이 살지 않아, 저장공간의 80~90%를 압축할 수 있음.
책에서는 50PB로 가정.

각 확대 수준에 따른 타일들까지 포함하면 일주 67PB < 100PB

이미지에 비해서 도로 정보는 훨씬 가벼움.
책에서는 미가공 도로 데이터가 수TB 용량이라고 가정하였음.

2024년 기준 서울의 도로 총 길이는 8300km 정도, 2012년 기준 고속도로 및 간선도로가 1400km
일주 50m에 교차로가 하나 있어도 edge가 16만개.
경로 탐색을 위해 필요한 정보가 edge하나가 평균 1KB 정도 한다고 가정하면
서울의 도로정보만 올리는건 양방향 도로 32만개 * 1KB = 300MB정도 되나?
인도나 자전거도로, 기타 대중교통 정보를 포함하더라도 단일 머신에 올리는데 지장없을듯

서버 대역폭

타일 전송이 대부분

서버가 처리해야하는 요청은 크게 두가지
경로 안내 요청과 사용자 위치 변경 요청

DAU 10억의 사용자가 경로 안내 기능을 주당 35분 정도 사용한다고 가정,
하루에 50억 분이 길찾기 서비스 사용시간이 발생함.

길찾기 서비스 사용도중 15초에 한번씩 사용자의 GPS 위치 갱신이 발생한다고 가정해서
위치 갱신 QPS는 20만 정도가 됨

국내에서 사용하는게 대부분이다보니 10억 DAU에 도로망 데이터 전세계를 가정한게 이해가 되진 않았음
구글지도 전세계적으로는 실시간 길찾기로도 자주 사용하나? 버스 현재 위치 안보여주거나 도로 포화정도 안나와서 답답하던데
빨리빨리의 민족이라 신경쓰이나 싶기도

개략적 설계안

크게 세가지 파트로 구성됩니다

- 위치 서비스 (location service)
- 경로 안내 서비스 (navigation service)
- 지도 표시 (map rendering)

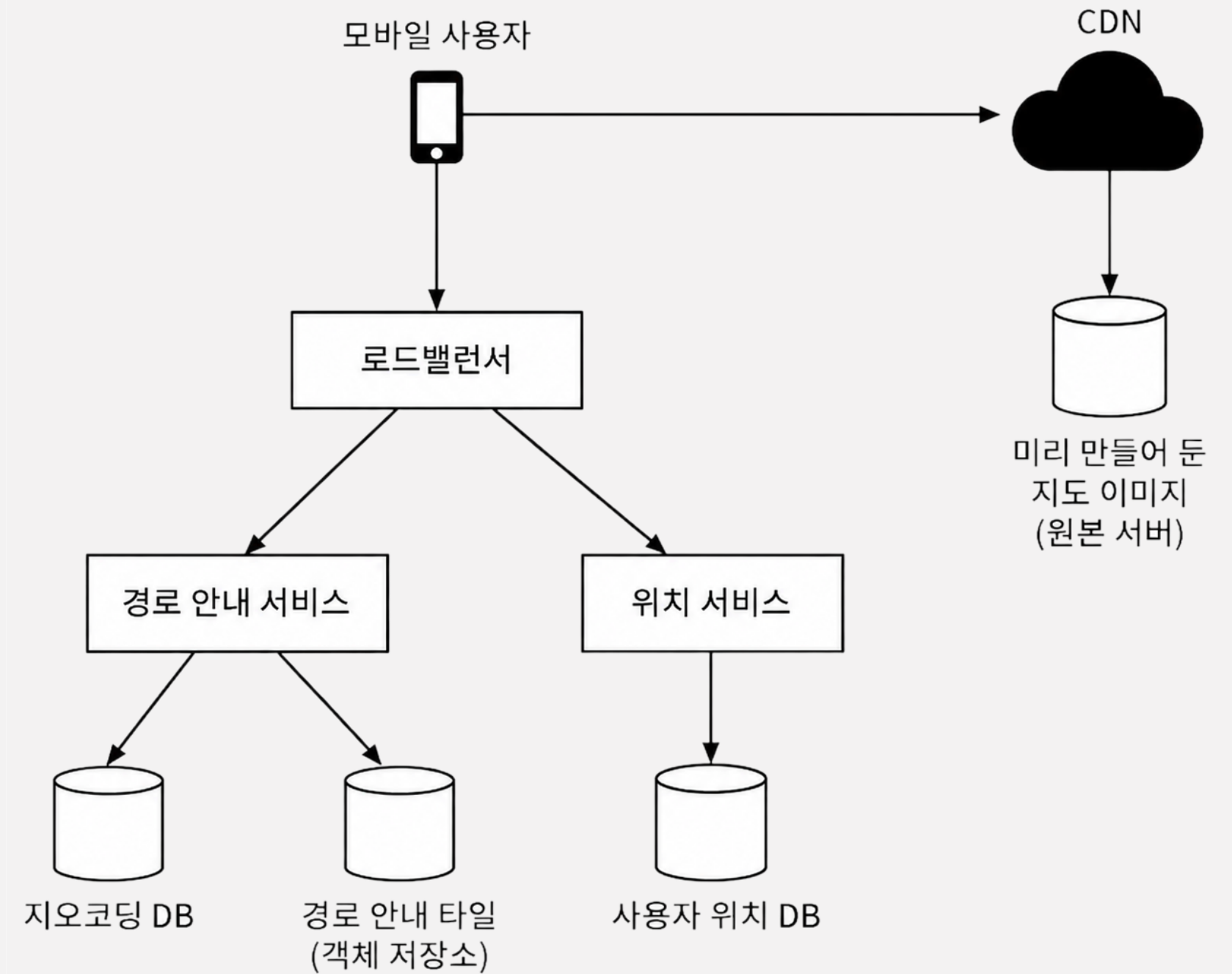


그림 3.7 개략적 설계안

위치 서비스

교통상황 모니터링
새 도로나 폐쇄된 도로 탐지
개인화된 경험 제공
실시간 위치 / 위치 변화에 따른 더 정확한 ETA 산출 및 경로 계산

기본 설계안은 t초마다 자기 위치를 전송한다고 가정합니다.

사용자의 위치 변화가 있을 때마다 기록하더라도, 위치 변경이력을 쌓아둔 뒤 일괄요청(batch)하면 전송 빈도를 줄일 수 있습니다.

구글맵과 같은 DAU 10억 서비스는 그럼에도 많은 쓰기 요청을 처리해야만 합니다.

따라서 아주 높은 쓰기 요청 빈도에 최적화되고 규모 확장이 용이한 카산드라(Cassandra)와 같은 DB가 필요합니다.

또한 카프카(Kafka)와 같은 스트림(Stream)처리 엔진을 활용해, 위치 정보를 로깅해야할수도 있습니다.

통신 프로토콜은 HTTP를 keep-alive 옵션과 함께 사용하면 효율을 높힐 수 있을 것입니다.

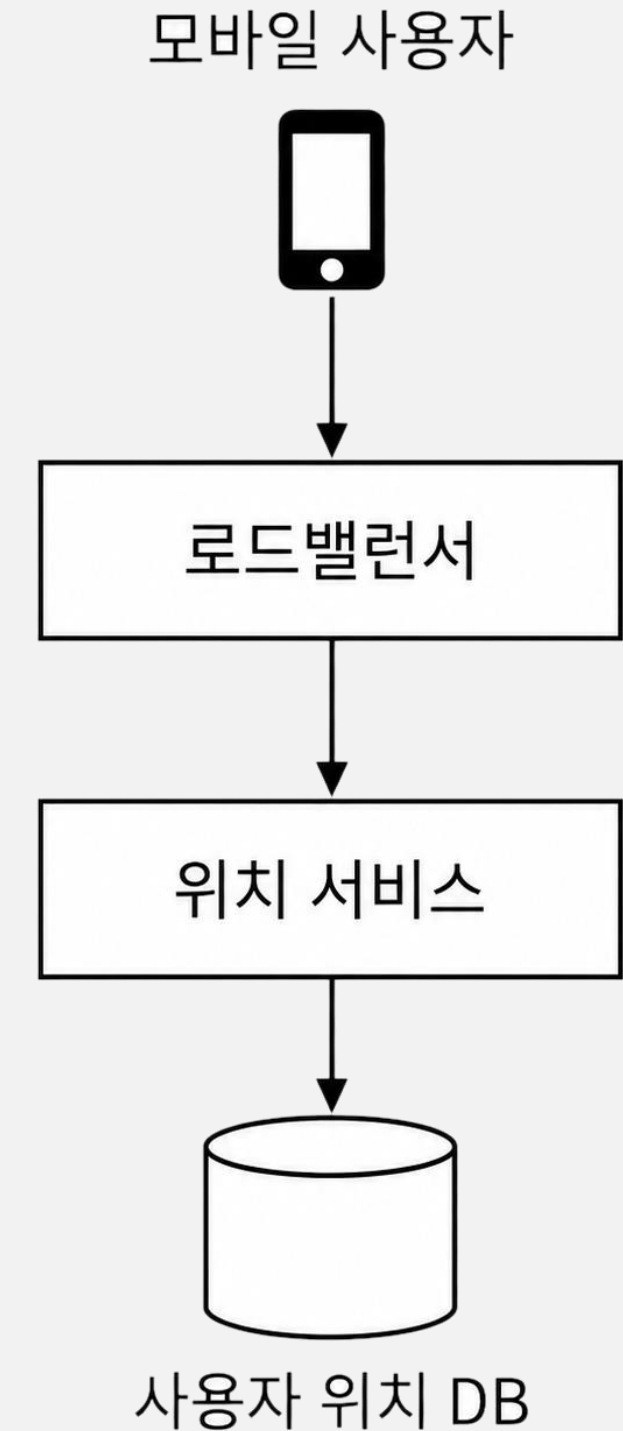


그림 3.8 위치 서비스

Cassandra : 단순한 형태의 데이터를 대량으로 쓰는데 최적화된 DB관리 시스템으로 이해하시면 됩니다
(복잡한 병합 / 임의 검색 / 강한 트랜잭션을 버리고 단일 데이터 쓰기에 최적화?)

Kafka : 발생한 이벤트들을 순서대로 기록하고 나중에 다른 서비스가 순서대로 꺼내 읽기 좋은 로깅에 최적화된 pub/sub 메시지 큐

관련지식이 많거나 직접 다뤄본 경험이 거의 없어서 그냥 설명을 위해 추가로 달아둠

경로 안내 서비스

A지점에서 B지점으로 가는 합리적으로 빠른 경로를 찾아주는 역할을 합니다. 결과를 얻는데 드는 시간지연은 어느정도 감내할 수 있고, 반드시 최단경로일 필요는 없으나 정확도는 보장되어야 합니다.

사용자가 보낸 경로 안내 HTTP요청은 로드밸런서를 거쳐 서비스에 도착합니다. 요청에는 출발지와 목적지가 전달되며, 결과는 다음과 같은 모습입니다. ->

경로 재탐색이나 교통상황 변화등은 상세 설계파트에서 알아보시다.

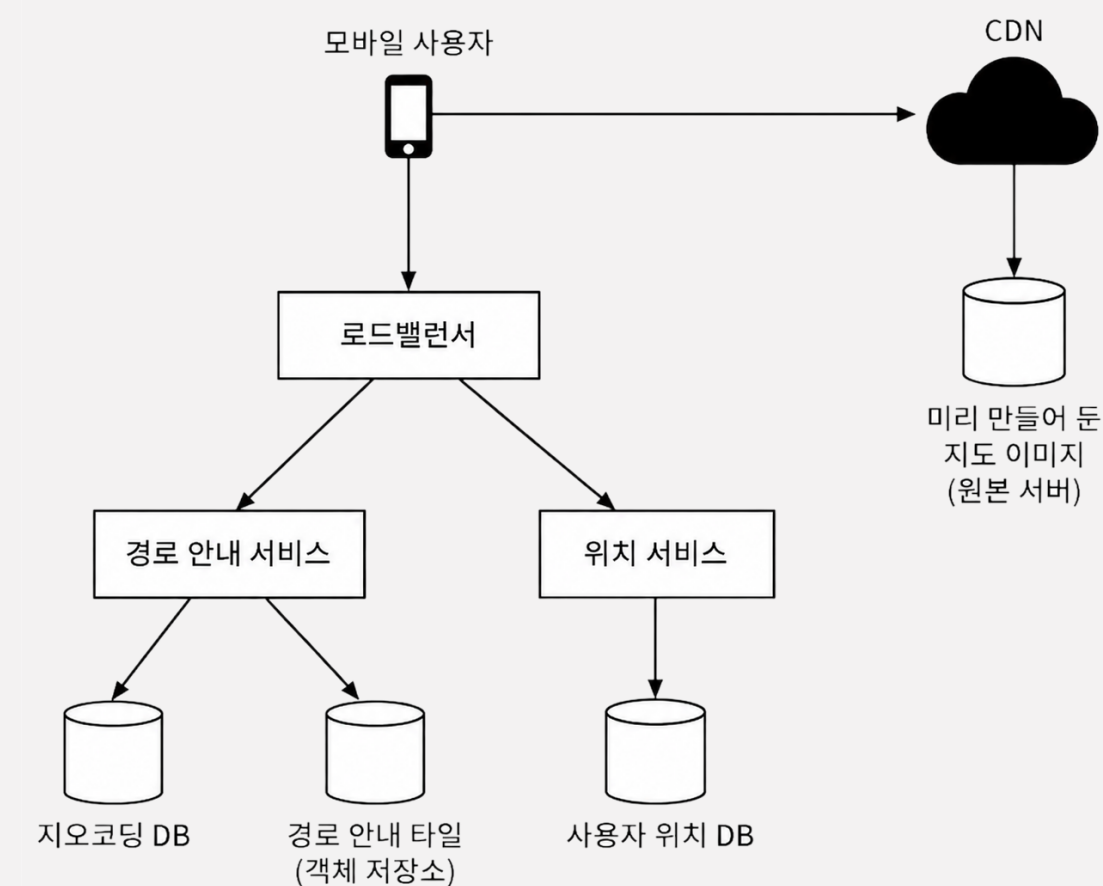


그림 3.7 개략적 설계안

```
{
  "status": "OK",
  "distance": {
    "text": "0.2 mi",
    "value": 259
  },
  "duration": {
    "text": "1 min",
    "value": 83
  },
  "start_location": {
    "lat": 37.4027165,
    "lng": -121.9435809
  },
  "end_location": {
    "lat": 37.4038943,
    "lng": -121.9410454
  },
  "html_instructions": "Head northeast on Brandon St toward Lumin Way",
  "polyline": {
    "points": "fbcfihbnVuAwDsCaL"
  },
  "geocoded_waypoints": [
    {
      "geocoder_status": "OK",
      "partial_match": true,
      "place_id": "ChIJvZNMti1fawwRO2aVVVX2yKg",
      "types": [
        "locality",
        "political"
      ]
    },
    {
      "geocoder_status": "OK",
      "partial_match": true,
      "place_id": "ChIJ3aPgQGtXawwRLYe1BMUI7bM",
      "types": [
        "locality",
        "political"
      ]
    }
  ],
  "travel_mode": "DRIVING"
}
```

지도 표시

클라이언트가 PB단위의 지도 이미지를 저장하고 있는건 현실적이지 않습니다. 클라이언트의 요청(위치, 확대 수준)에 따라 필요한 타일 이미지를 서버에서 가져오는 접근법이 바람직합니다.

클라이언트는 요청할 영역의 정보를 알고 있기 때문에, 각각의 지도 타일 요청을 CDN에 보낸 뒤 원본 서버에서 꺼내 저장한 사본을 반환받습니다.

규모 확장에도 용이하고 사용자와 인접해 트래픽도 줄일 수 있습니다. 지도 타일은 (대부분) 정적이기 때문에 아주 적합하며,

사용자의 클라이언트도 대부분 모바일이지만, 기기에도 지도 타일 정보를 캐시해둘 수 있어 초기 로드 이후에 데이터 사용량을 줄일 수 있습니다.

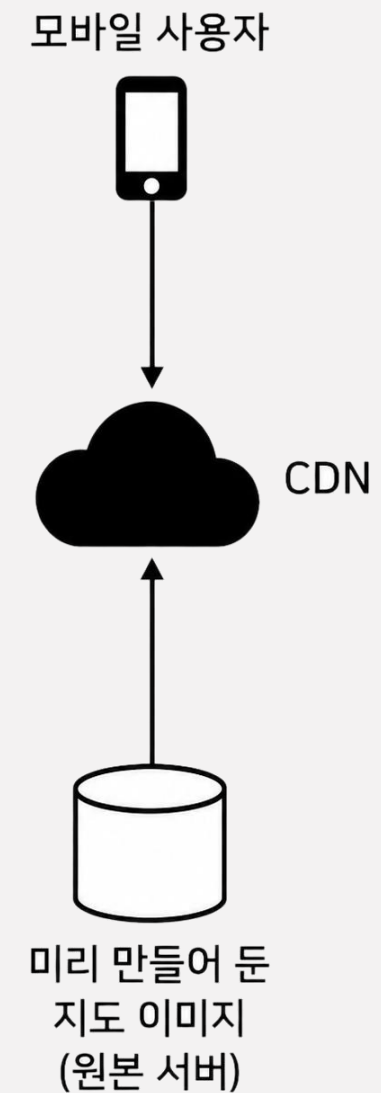


그림 3.10 CDN을 통한 사전 생성된 지도 이미지 서비스

지도 표시 - CDN에 요청을 보내는 방법

클라이언트가 어떻게 CDN에 바로 요청을 보낼 수 있을까요?
지도 타일은 지오 해시를 기반으로 생성되어 이미 정의된 격자에 맞게 확대 수준별로 하나씩 생성되어 있게 되므로 고유한 값을 가지며,
클라이언트에서도 동일하게 계산할 수 있게 됩니다.

하지만 지오해시 계산 알고리즘을 클라이언트에 구현하는 경우
지원해야하는 플랫폼이 많을 때 문제가 될 수 있습니다.
시간도 많이 걸리고 (검수) 때로는 위험한 프로세스이기 때문에

맵 타일 인코딩에 지오해싱을 사용할 것이라는 보장이 있어야만 합니다.

이를 해결하기 위한 다른 선택지는 옆의 이미지와 같이, 주어진 좌표/확대 수준을
입력받아 타일 URL을 계산하는 간단한 서비스를 배치하는 것입니다.

클라이언트가 CDN에 타일 지도 정보를 요청하는 것은 상세 설계에서 더 알아보겠습니다.

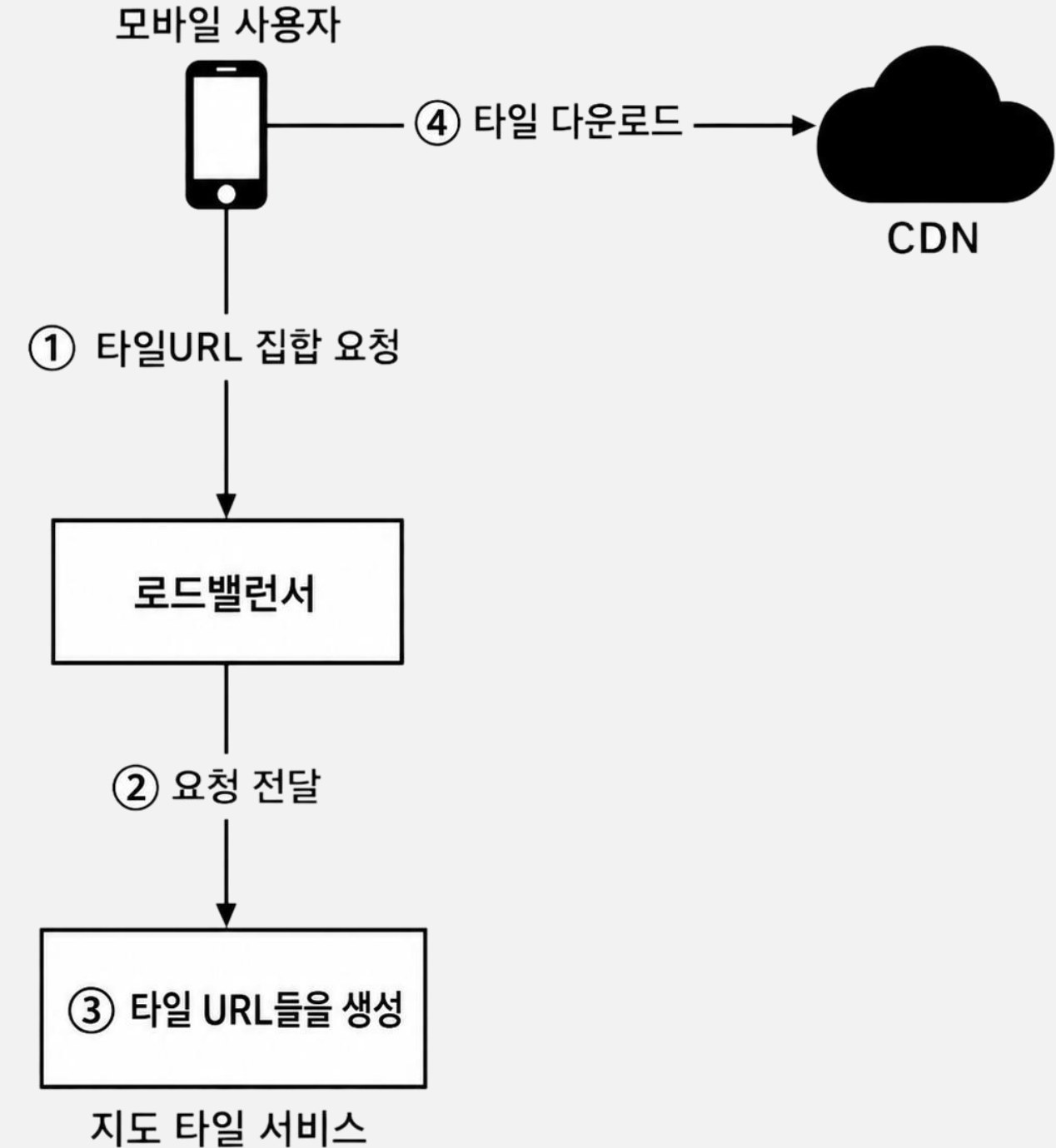


그림 3.12 지도 표시 흐름도

데이터 모델

서비스가 다루어야 할 데이터 모델을 먼저 살펴본 뒤, 각각의 서비스가 해당 데이터를 어떻게 다루어야 할지 설계합니다.

데이터	특징	저장소	목적
경로 안내 타일	배치 생성, 읽기 위주	오브젝트 스토리지	계층·지역 단위 파일
사용자 위치 데이터	대량의 쓰기연산, 시계열	쓰기 최적화 DB	대량 위치 갱신 수집
지오코딩 DB	조회 위주	키-값 저장소	주소 ↔ 좌표
미리 만든 지도 이미지	정적, 대용량	오브젝트 스토리지 + CDN	줌 레벨별 타일 이미지

경로 안내 타일

필요한 데이터는 외부 사업자나 기관이 제공한 것을 이용 한다고 가정하였으며 (수 TB) 애플리케이션이 지속적으로 수집한 사용자 위치 데이터를 통해 개선 될 수 있습니다.

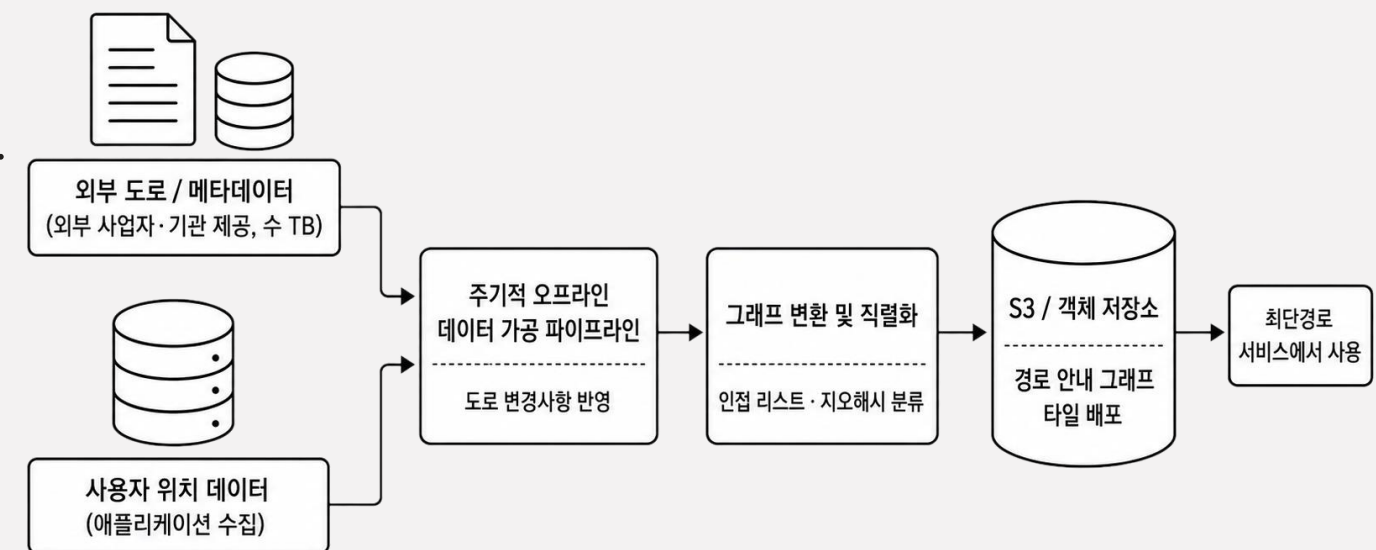
이 데이터는 방대한 양의 도로(인도, 자전거 도로, 대중교통등 포함)와 메타데이터로 이루어져 있으며 그래프 자료구조로 이루어진 상태가 아니라 경로 안내 알고리즘의 입력으로 사용할 수 없습니다.

주기적으로 경로 안내 알고리즘이 사용할 수 있는 형태로 가공하는 오프라인 데이터 가공 파이프라인을 실행하여 도로 데이터에 발생한 변경사항을 반영하여 이를 해결할 수 있습니다.

그래프의 노드와 선을 데이터베이스 레코드로 저장하는 것도 방법일 수 있지만, 비용이 많이 들고, DB가 제공하는 기능들을 거의 사용할 필요가 없기 때문에 비효율적입니다.

인접리스트 형태의 그래프를 S3같은 객체 저장소(Object storage)에 직렬화 해서 저장하고, 지오해시 기준으로 분류해 필요한 데이터를 쉽게 찾을 수 있게 구성할 수 있습니다.

이 데이터를 사용해 최단경로 서비스를 구축하는 것은 서비스 설계에서 다루게 됩니다.



경로 안내 타일 데이터 가공 및 S3 배포 개요

* S3 / Object storage : 폴더가 없는 파일 저장소로 이해하면 쉽습니다. 파일을 찾기 위한 식별자가 있으면 훨씬 쉽게 찾을수 있고, 확장성이 더 좋습니다.

사용자 위치 데이터

사용자의 위치 정보는 아주 값진 데이터입니다.

user id / timestamp / user state / location을 저장하여

도로 데이터 및 경로 안내 타일을 갱신하고

실시간 교통 상황 데이터 / 교통 상황 데이터 이력 DB를 구축하는데에도 활용됩니다.

사용자 위치 데이터를 저장하려면 엄청난 양의 쓰기 연산을 잘 처리할 수 있으며, 수평적
규모 확장이 가능해야하기 때문에 카산드라(Cassandra)를 사용해 저장하게 될 것
같습니다.

지오코딩 데이터베이스

주소를 위/경도 좌표로 변환하는 정보를 보관합니다.

해당 정보는 변경 (쓰기) 연산이 드물게 발생하기 때문에
Redis와 같이 빠른 읽기 연산을 제공하는 키-값 저장소를 사용하는 것이 좋겠습니다.

경로 계획 서비스에서 출발지와 목적지를 주소로 처리하기엔 어려움이 있기 때문에,
지오코딩 DB를 통해 위/경도 좌표로 변환하여 경로 계획 서비스에 전달해주어야 합니다.

미리 만들어 둔 지도 이미지

클라이언트에서 특정 영역의 지도를 요청하면 인근 도로 정보와 상세 정보가 포함된 이미지를 생성해야 합니다. 생성시 계산 자원을 많이 사용하며, 중복 요청이 자주 들어오게 되기 때문에 한번만 생성하고 캐시해두는 전략이 매우 유효합니다.

개략적 설계안에서 다루었듯, 확대(줄)수준에 따라 미리 이미지를 만들어두고 요청이 들어온 CDN을 통해 전송합니다.

원본은 S3같은 오브젝트 저장소를 사용하면 되겠습니다.

서비스

서비스가 다루어야 할 데이터 모델을 먼저 살펴본 뒤, 각각의 서비스가 해당 데이터를 어떻게 다루어야 할지 설계합니다.

서비스

특징

위치 서비스

가치있는 정보를 포함한 대량 쿼리의 진입점

지도 표시

대량의 읽기 연산, 클라이언트 기반의 요청

경로 안내 서비스

고려해야할 데이터가 많아 처리과정이 복잡

위치 서비스

개략적 설계안에서는 위치 서비스 동작 방식에 대해 논의했었습니다.

사용자의 최신 위치 갱신(user id / timestamp / user state / location)요청에 대한 쓰기연산은 카산드라 / 로깅은 카프카
상세 설계에서는 사용자의 위치정보를 어떻게 이용하는지에 초점을 맞추어 설계해봅시다.

user id는 파티션 키 (Partition Key)

- 특정 사용자의 위치를 조회하기 위함

timestamp는 클러스터링 키 (Clustering Key)

- 시간순서대로 정렬하여 최신 / 또는 특정 기간내 위치를 효율적으로 탐색하기 위함

사용자 위치 데이터는 카프카에 기록해서

- 실시간 교통 상황도 갱신
- 경로 안내 타일 처리시에는 도로상태를 탐지하여 품질을 개선
- 기타 데이터 기록 / 처리

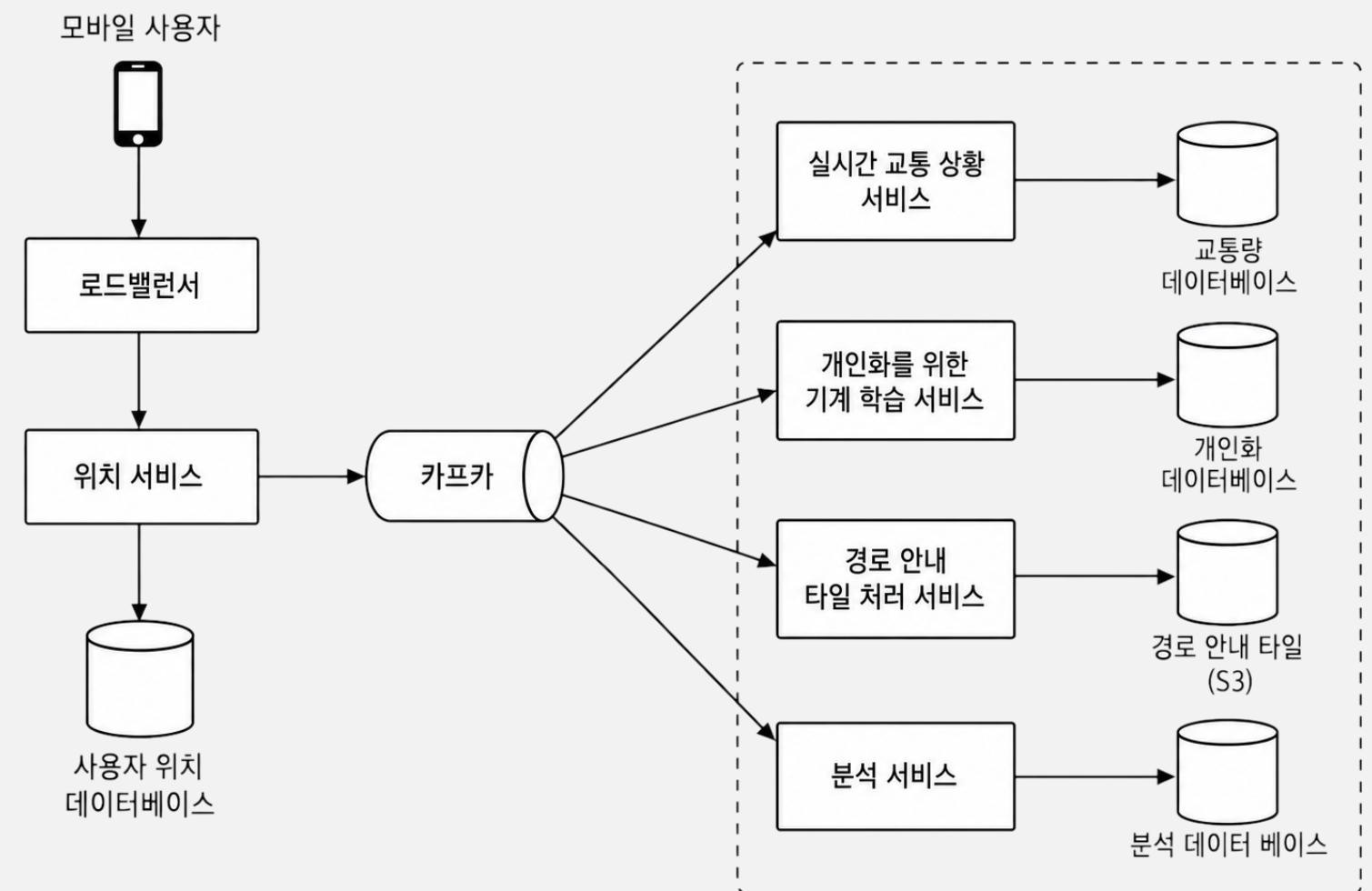


그림 3.15 여러 서비스에 위치 데이터 제공

지도 표시

개략적 설계에서 다룬 내용대로 클라이언트가 보이는 영역의 타일 URL을 계산해 요청

- 확대 정도가 변하더라도 요청해야하는 타일의 개수는 거의 유사함

클라이언트 사이드에서 렌더링하기

- 256x256이미지가 아니라 경로/다각형과 같은 vector정보를 보낸뒤, 클라이언트 사이드에서 WebGL 기술을 통해 렌더링하면 훨씬 대역폭을 아낄 수 있고, 매끄러운 지도 확대 경험을 구현할 수 있음.
- 특정 스팟의 정보 같은게 상호작용 가능하게 띄우기 수월해 1장에서 다룬 인접성 서비스와도 결합가능

경로 안내 서비스

지오코딩 서비스를 통해 경로 계획 서비스가 사용할 수 있는 위도/경도 쌍 생성

최단 경로 서비스는 도로나 교통상황을 고려하지 않고 정적인 도로 상황에서
경로 안내 타일 정보를 참고해 그래프 기반의 top-k 최단경로를 탐색

현재/과거 교통상황에 근거하여 경로에 대한 예측 시간을 계산

계산된 경로별 예측 시간과 필터링 서비스를 통해 최적 경로 출력

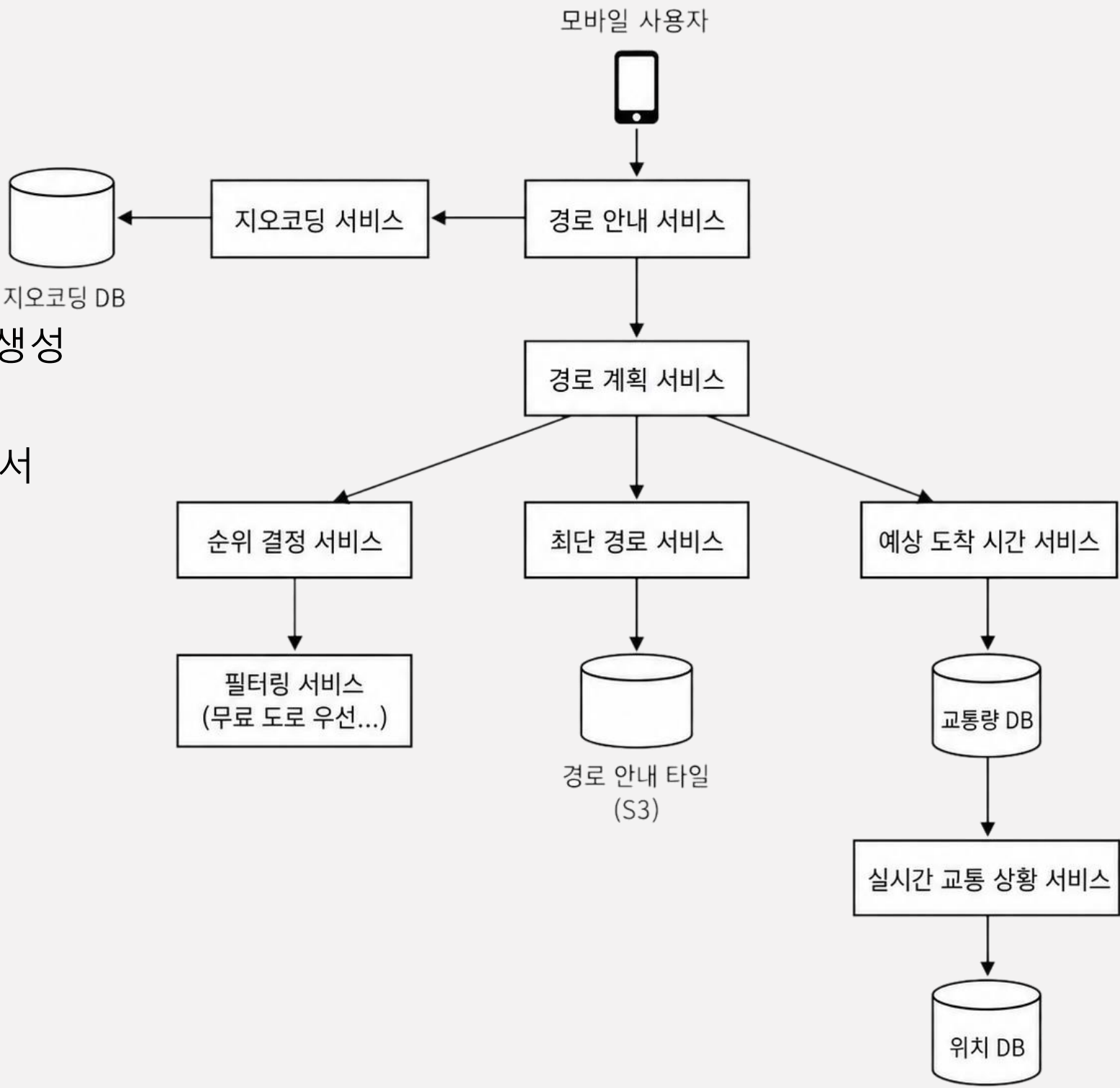


그림 3.17 경로 안내 서비스

최종 정리

측위 -> 도법 -> 지오코딩으로 사용자 위치 정보를 다루기

지오해싱으로 지도는 타일로 나눠 미리 렌더링 (또는 벡터 데이터로 가공)하고 CDN으로 전달

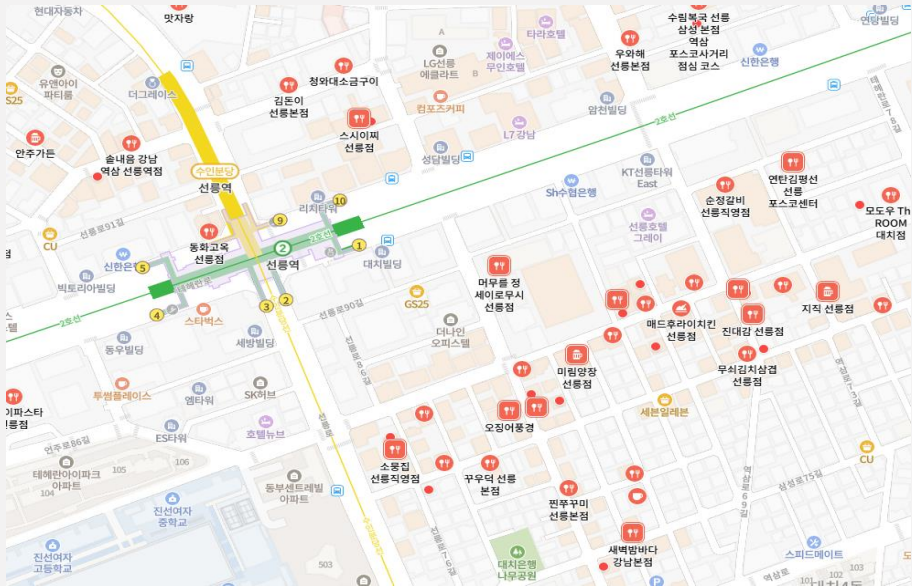
경로 안내는 도로 그래프를 계층적 라우팅 타일로 분할해 빠르게 탐색

경로 안내 타일 / 사용자 위치 / 지오코딩 / 지오해싱 기반의 지도 타일의
데이터 4종을 각각의 성격에 맞는 저장소에 나눠 담고 필요한 서비스에서 사용하기

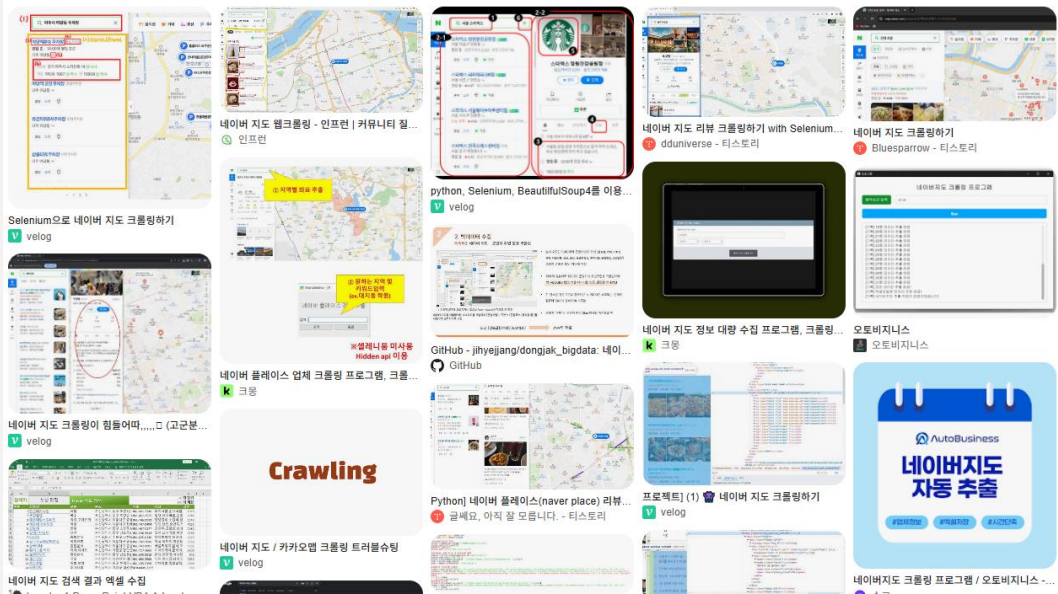
나아가서...



지도 데이터 확보가 어려울 경우 어떻게 할까?



대부분의 지도는 근접성 서비스도 함께 다룬다.
이때 생길 수 있는 고충이 있을까?



대 시시대, 열심히 가공한 데이터의 크롤링은 어떻게 방지할까
아니면 시가 사용할 수 있는 형태로 제공할 방법은 무엇이 있을까?

감사합니다

부족한 자료는 나중에 보충하여 업로드 하겠습니다
앞에 궁금한 내용들 보다보니 핵심 후반부가 힘이 빠지고 지엽적인 지식들이 더 생김...